```
!pip install -q groq requests
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 143.7/143.7 kB 1.3 MB/s eta 0:00:00
```

```
from google.colab import userdata
from groq import Groq
import requests
import sqlite3
import json

Secret_Key = userdata.get("Secret_Key")

if Secret_Key is None:
    print("Secret_Key not found")
else:
    print("Secret_Key loaded successfully")

client = Groq(
    api_key=Secret_Key.strip()
)

print("Groq client created successfully")
```

```
Secret_Key loaded successfully
Groq client created successfully
```

```
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[
        {
            "role": "user",
            "content": "Say hello in one short sentence."
        }
    ]
)

print(response.choices[0].message.content)
```

```
Hello.
```

```
# ---------------- CALCULATOR ----------------

def safe_calculator(expression):

    expression = expression.replace("×", "*")
    expression = expression.replace("x", "*")
    expression = expression.replace("X", "*")

    allowed = "0123456789+-*/(). "

    if any(char not in allowed for char in expression):
        return "Invalid mathematical expression"

    try:
        return eval(expression, {"__builtins__": None}, {})
    except:
        return "Unable to calculate"


# ---------------- DATABASE ----------------

conn = sqlite3.connect("students.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    name TEXT,
    marks INTEGER
)
""")

students = [
    ("Amit", 85),
    ("Rahul", 78),
```

```
    ("Sneha", 92),
    ("ABC", 20),
    ("PQR", 22)
]

cursor.executemany(
    "INSERT OR IGNORE INTO students VALUES (?, ?)",
    students
)

conn.commit()
conn.close()


def database_search(name):

    conn = sqlite3.connect("students.db")

    cursor = conn.cursor()

    cursor.execute(
        "SELECT * FROM students WHERE name=?",
        (name,)
    )

    result = cursor.fetchone()

    conn.close()

    return result


# ---------------- WEATHER API ----------------

def weather_api(city):

    geo_url = "https://geocoding-api.open-meteo.com/v1/search"

    geo_params = {
        "name": city,
        "count": 1,
        "language": "en",
        "format": "json"
    }

    response = requests.get(
        geo_url,
        params=geo_params
    )

    if response.status_code != 200:
        return "Unable to find city"

    data = response.json()

    if "results" not in data:
        return "City not found"

    latitude = data["results"][0]["latitude"]
    longitude = data["results"][0]["longitude"]
    city_name = data["results"][0]["name"]

    weather_url = "https://api.open-meteo.com/v1/forecast"

    weather_params = {
        "latitude": latitude,
        "longitude": longitude,
        "current": "temperature_2m,relative_humidity_2m,wind_speed_10m"
    }

    weather_response = requests.get(
        weather_url,
        params=weather_params
    )

    if weather_response.status_code != 200:
        return "Weather API failed"

    weather_data = weather_response.json()

    current = weather_data["current"]
```

```
    return {
        "city": city_name,
        "temperature": current["temperature_2m"],
        "humidity": current["relative_humidity_2m"],
        "wind_speed": current["wind_speed_10m"]
    }


# ---------------- WEB SEARCH ----------------

def web_search(query):

    from bs4 import BeautifulSoup

    url = "https://html.duckduckgo.com/html/"

    params = {
        "q": query
    }

    headers = {
        "User-Agent": "Mozilla/5.0"
    }

    response = requests.get(
        url,
        params=params,
        headers=headers,
        timeout=10
    )

    if response.status_code != 200:
        return "Web search failed"

    soup = BeautifulSoup(
        response.text,
        "html.parser"
    )

    results = []

    for item in soup.select(".result")[:5]:

        title_tag = item.select_one(".result__a")

        if title_tag:

            title = title_tag.get_text(
                " ",
                strip=True
            )

            link = title_tag.get("href")

            results.append({
                "title": title,
                "link": link
            })

    if not results:
        return "No search results found"

    return results
```

```
def select_tool(query):

    prompt = f"""
You are the decision-making component of an autonomous agent.

The agent has access to these tools:

1. calculator
2. database
3. weather_api
4. web_search

User query:
{query}

Choose the most appropriate tool.

Rules:
```

```
calculator:
Use for mathematical calculations.

database:
Use for student names, marks, or student records.

weather_api:
Use for weather, temperature, humidity, wind, or city weather.

web_search:
Use for general information or search requests.

Return ONLY the tool name.
"""

    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[
            {
                "role": "system",
                "content": "You are a tool-selection agent."
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
        temperature=0
    )

    return response.choices[0].message.content.strip().lower()
```

```
def autonomous_agent(query):

    print("\n==============================")
    print("AUTONOMOUS AGENT")
    print("==============================")

    print("User Query:", query)

    # Step 1: Decide which tool to use

    tool = select_tool(query)

    print("Selected Tool:", tool)

    # Step 2: Execute the selected tool

    if tool == "calculator":

        prompt = f"""
Convert this user query into a mathematical expression.

User query:
{query}

Examples:

add 35 and 67
35 + 67

multiply 14 and 56
14 * 56

multiply 56 and 78 and then add 12
56 * 78 + 12

Return ONLY the mathematical expression.
Do not return JSON.
Do not return explanation.
"""

        try:
            response = client.chat.completions.create(
                model="llama-3.3-70b-versatile",
                messages=[
                    {
                        "role": "user",
                        "content": prompt
                    }
                ],
```

```
                temperature=0
            )
            expression = response.choices[0].message.content.strip()
        except Exception as e:
            print(f"Error fetching expression from LLM: {e}")
            expression = "" # Default to empty string if API call or parsing fails

        print("Expression:", expression)

        result = safe_calculator(expression)


    elif tool == "database":

        prompt = f"""
Extract the student name from this query:

{query}

Return ONLY the name.
"""

        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            temperature=0
        )

        name = response.choices[0].message.content.strip()

        result = database_search(name)

        if result is None:
            result = "Student not found"


    elif tool == "weather_api":

        prompt = f"""
Extract the city name from this query:

{query}

Return ONLY the city name.
"""

        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            temperature=0
        )

        city = response.choices[0].message.content.strip()

        result = weather_api(city)


    elif tool == "web_search":

        result = web_search(query)


    else:

        result = "No suitable tool found"


    # Step 3: Return result

    print("Tool Result:", result)

    return result
```

```
while True:

    query = input("\nEnter your query or type exit: ")

    if query.lower() == "exit":
        print("Agent stopped")
        break

    try:

        result = autonomous_agent(query)

        print("\nFinal Answer:", result)

    except Exception as e:

        print("\nError:", e)
```

```
Enter your query or type exit: 25 multiply by 16 and add 56

==============================
AUTONOMOUS AGENT
==============================
User Query: 25 multiply by 16 and add 56
Selected Tool: calculator
Expression: 25 * 16 + 56
Tool Result: 456

Final Answer: 456

Enter your query or type exit: what are amits marks

==============================
AUTONOMOUS AGENT
==============================
User Query: what are amits marks
Selected Tool: database
Tool Result: ('Amit', 85)

Final Answer: ('Amit', 85)

Enter your query or type exit: what is todays weather in Pune

==============================
AUTONOMOUS AGENT
==============================
User Query: what is todays weather in Pune
Selected Tool: weather_api
Tool Result: {'city': 'Pune', 'temperature': 22.7, 'humidity': 90, 'wind_speed': 13.1}

Final Answer: {'city': 'Pune', 'temperature': 22.7, 'humidity': 90, 'wind_speed': 13.1}

Enter your query or type exit: what is Generative AI

==============================
AUTONOMOUS AGENT
==============================
User Query: what is Generative AI
Selected Tool: web_search
Tool Result: [{'title': 'What is generative AI? - IBM', 'link': '//duckduckgo.com/l/?uddg=https%3A%2F%2Fwww.ibm.com%2Fthink%

Final Answer: [{'title': 'What is generative AI? - IBM', 'link': '//duckduckgo.com/l/?uddg=https%3A%2F%2Fwww.ibm.com%2Fthink

Enter your query or type exit: exit
Agent stopped
```

Start coding or generate with AI.